then we can traverse the map, the option, and the list at the same time and easily get to the `V` value inside, because `Map`, `Option`, and `List` are all traversable.

EXERCISE 12.19

Implement the composition of two `Traverse` instances.

```
def compose[G[_]](implicit G: Traverse[G]):
    Traverse[({type f[x] = F[G[x]]})#f]
```

12.7.6 Monad composition

Let's now return to the issue of composing monads. As we saw earlier in this chapter, `Applicative` instances always compose, but `Monad` instances do not. If you tried before to implement general monad composition, then you would have found that in order to implement `join` for nested monads `F` and `G`, you'd have to write something of a type like `F[G[F[G[A]]]] => F[G[A]]`. And that can't be written generally. But if `G` also happens to have a `Traverse` instance, we can `sequence` to turn `G[F[_]]` into `F[G[_]]`, leading to `F[F[G[G[A]]]]`. Then we can join the adjacent `F` layers as well as the adjacent `G` layers using their respective `Monad` instances.

EXERCISE 12.20

Hard: Implement the composition of two monads where one of them is traversable.

```
def composeM[F[_],G[_]](F: Monad[F], G: Monad[G], T: Traverse[G]):
    Monad[({type f[x] = F[G[x]]})#f]
```

Expressivity and power sometimes come at the price of compositionality and modularity. The issue of composing monads is often addressed with a custom-written version of each monad that's specifically constructed for composition. This kind of thing is called a *monad transformer*. For example, the `OptionT` monad transformer composes `Option` with any other monad:

```
case class OptionT[M[_],A](value: M[Option[A]])(implicit M: Monad[M]) {
  def flatMap[B](f: A => OptionT[M, B]): OptionT[M, B] =
    OptionT(value flatMap {
      case None => M.unit(None)
      case Some(a) => f(a).value
    })
}
```

`Option` is added to the inside of the monad `M`.